Combinational Logic – Part 2

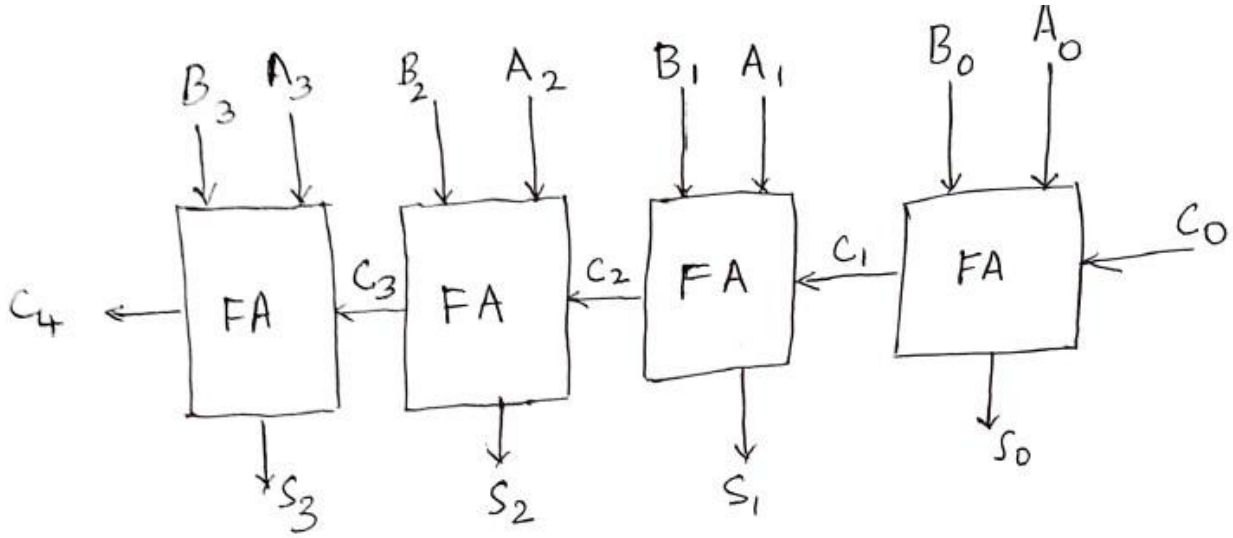
K. Sridharan

Review

- We have discussed adders.
- We will now discuss a few more aspects relating to multi-bit addition.
- Then we will look at other combinational logic circuits.

Drawback of ripple carry addition (RCA)

- We have carry propagation from the right end all the way to the left. This leads to considerable delay in producing the final output. See diagram:

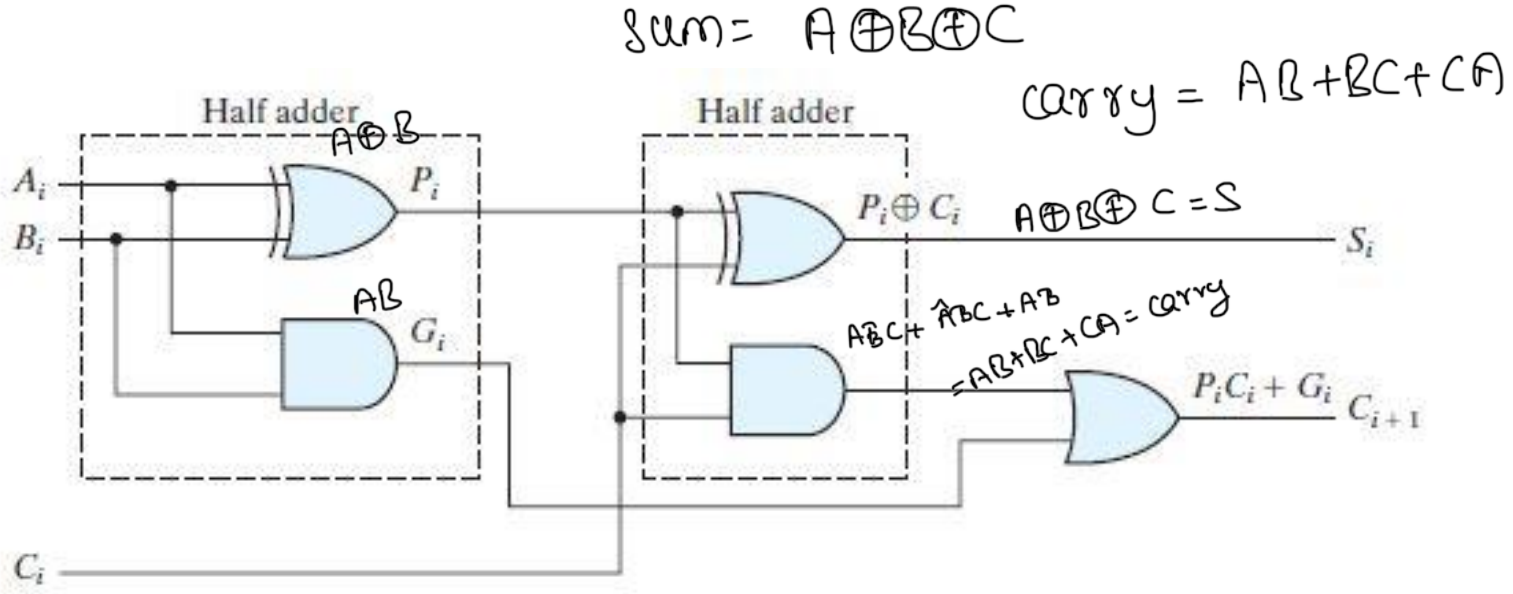


More on delay in RCA

- Signals must propagate through the gates before the correct sum can produced at the output terminals.
- The total propagation time depends on the propagation delay of a typical gate as well as the number of gate levels in the circuit.
- The longest propagation delay time in an adder corresponds to the time it takes for the carry to propagate through the full adders.

The concept of carry lookahead

Can we compute the carries in parallel ? That is, can we find C_1 , C_2 , C_3 etc. in parallel ? We can by introducing some extra hardware. We begin by considering a full adder.



Nothing has changed so far !

- We have only introduced two symbols, namely P_i and G_i in the full adder circuit

OR is as good as XOR for carry

- By the way, we can define the propagate using OR as well (since the carry equation is given by $C_1 = AB + BC_0 + C_0A$) so propagate signal can be simply defined as OR(A,B)
- When we do this, the sum has to be defined differently. We will discuss this next.

How is the sum defined ?

- The sum can be defined as G xor P xor C when we use P = A + B
- Exercise: Verify this.

How to get carries in parallel ?

We have

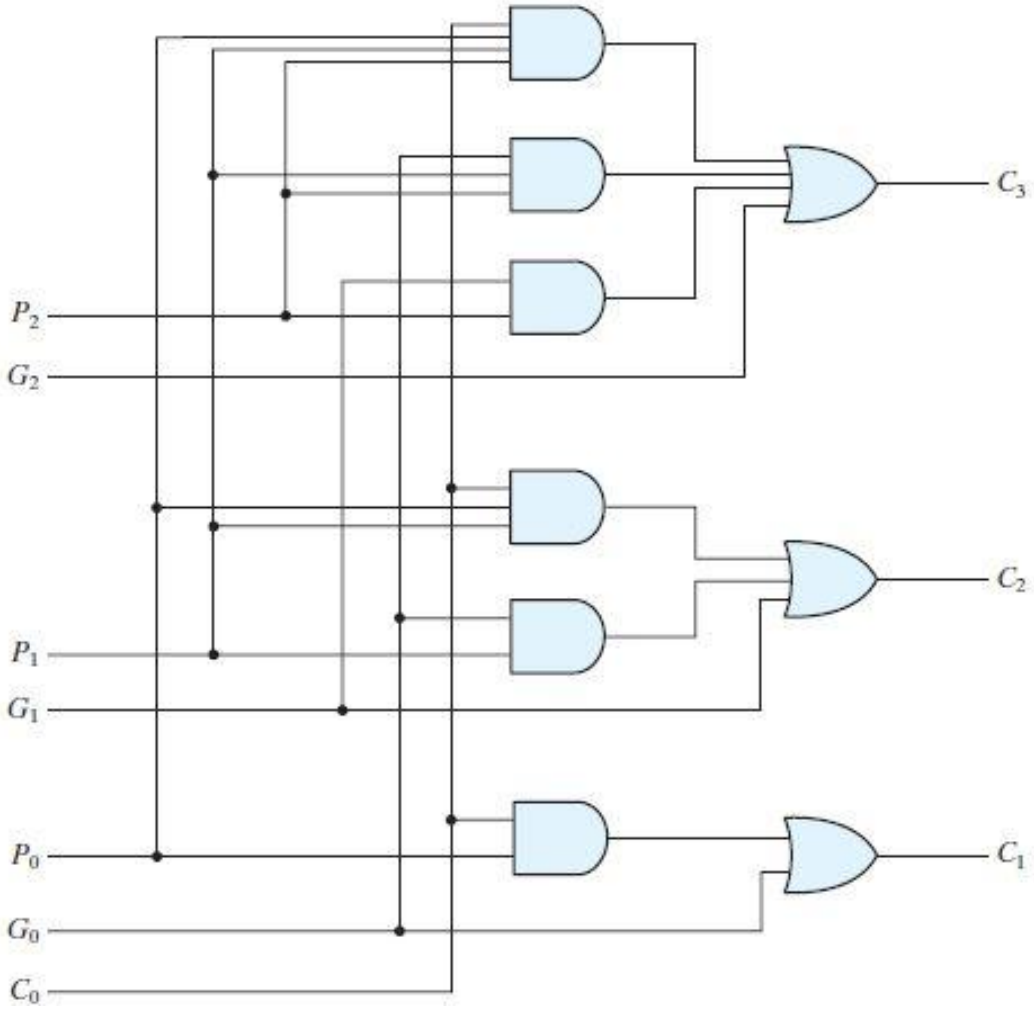
$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

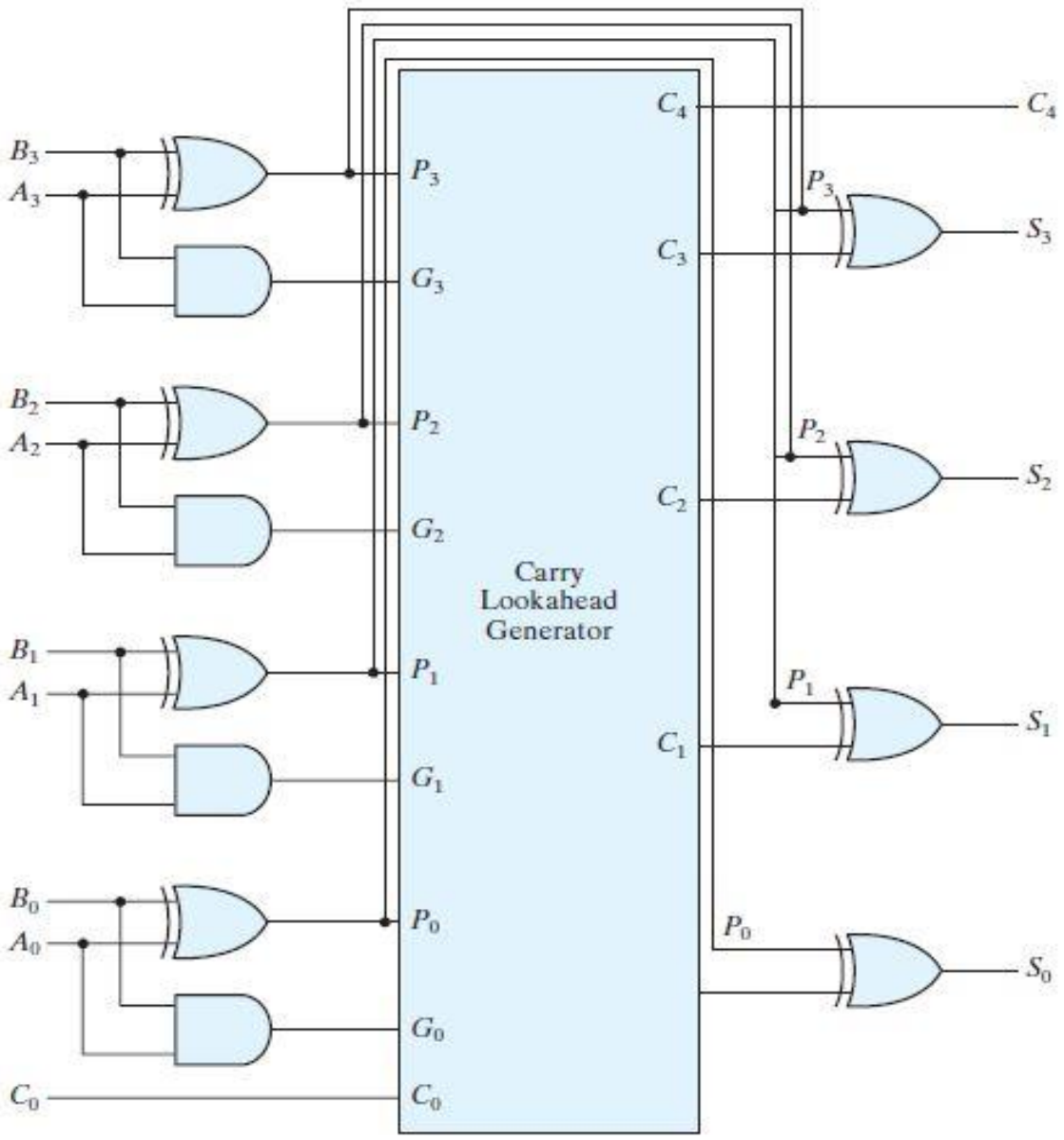
……

and so on.

How do we produce the carries from just the input carry and generate and propagate signals ?



Four-bit adder with carry lookahead



Explanation

- The 4-bit adder circuit with carry lookahead uses the carry lookahead generator shown earlier (that produces the various output carries from the input carry C_0 and the generate plus propagate signals).
- Note that all the carry outputs are simultaneously produced.

Is there any difficulty with this approach ?

- We mentioned that the carry lookahead approach allows us to speed up the addition process (by reducing the carry propagation time).
- Are there any concerns ? Question for students.

Consider the equations again...

$P_i = A_i \oplus B_i$

$G_i = A_i B_i$

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

Number of inputs to a gate

- As we move towards higher order carries from the “rightmost carry”, we see that the number of inputs to some gates becomes high.
- Normally, there are limits on the number of inputs that a gate can handle. This is referred to as fan-in.
- What will high fan-in actually do ?

What will high fan-in actually do ?

- When number of inputs to a gate becomes high, the signal propagation delay through the gate will increase. So the performance will be poor.
- Moreover, high fan-in can lead to signal degradation and reduce the reliability of the circuit.

So what do we do now ?

- When fan-in limitations exist, we need to redesign the arrangement.
- That is, we need to ensure that the number of inputs a gate can handle (without any problems) is met.
- One possibility is a hierarchical design. This is indicated next (we will only outline a solution, no details of the modified circuits are required for now).

One solution to address the fan-in problem

- A 16-bit carry lookahead adder can be implemented using four 4-bit carry lookahead circuits.
- Similarly, a 64-bit carry lookahead adder can take advantage of 16-bit circuits and so on.

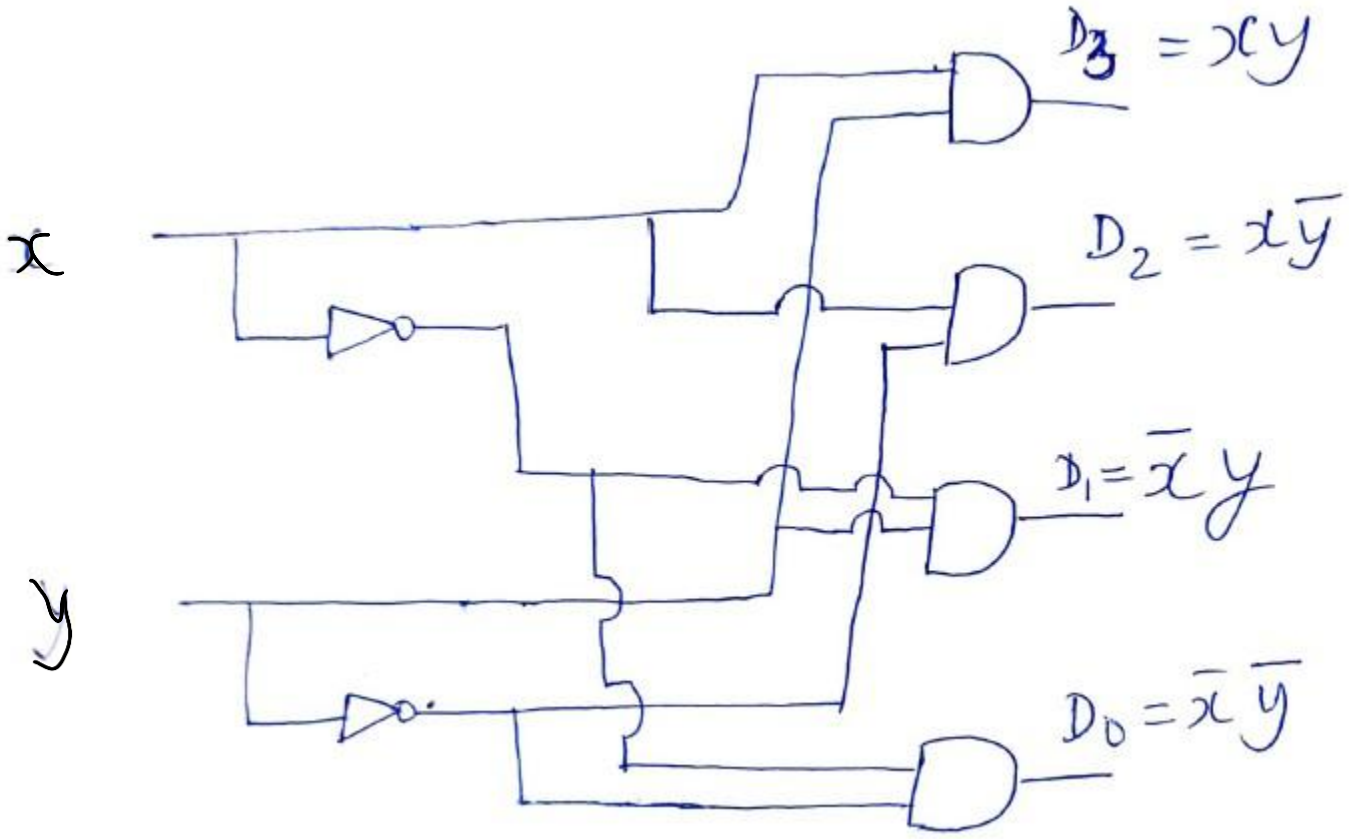
Beyond addition ...

- We now move on to other combinational circuits.
- In particular, we look at decoders here.

Decoder

- Decoder is a combinational circuit to convert binary information from n input lines to a maximum of 2^n unique output lines.
- Each combination of inputs will assert a unique output.
- In general, a decoder is a code converter.
- Decoders can be used to implement arithmetic functions as we will see soon.

2-to-4 decoder



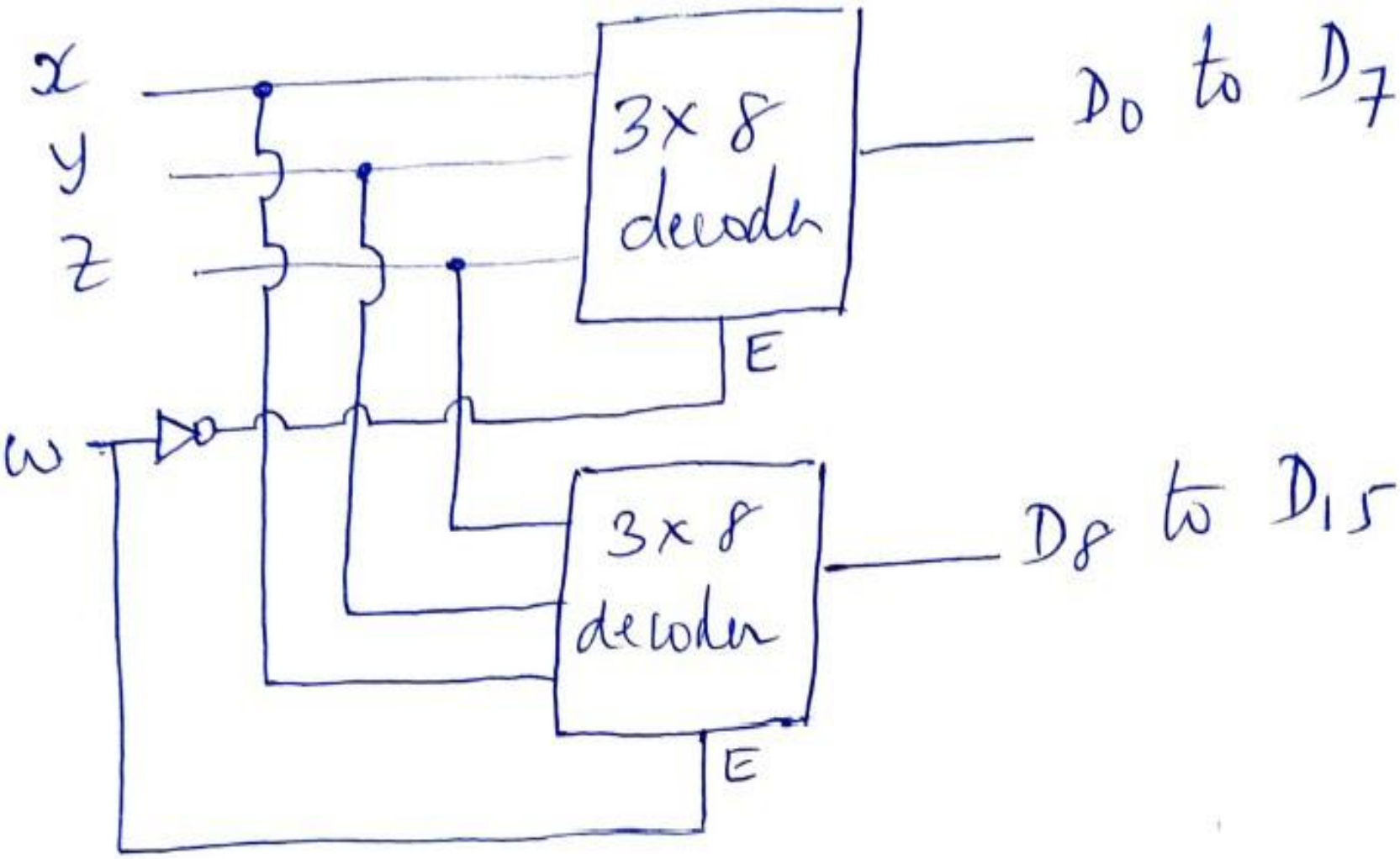
Truth table of a 2-to-4 decoder

Inputs		Outputs			
x	y	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Can we not have larger decoders ?

- Yes, we can have a 3-to-8 decoder.
- Exercise: draw the circuit of a 3-to-8 decoder
- We can also have a 4-to-16 decoder.
- In fact, we can use small decoders to construct a large decoder. For example, a pair of 3-to-8 decoders can be used to realize a 4-to-16 decoder as shown next.

4-to-16 decoder via 3-to-8 decoders



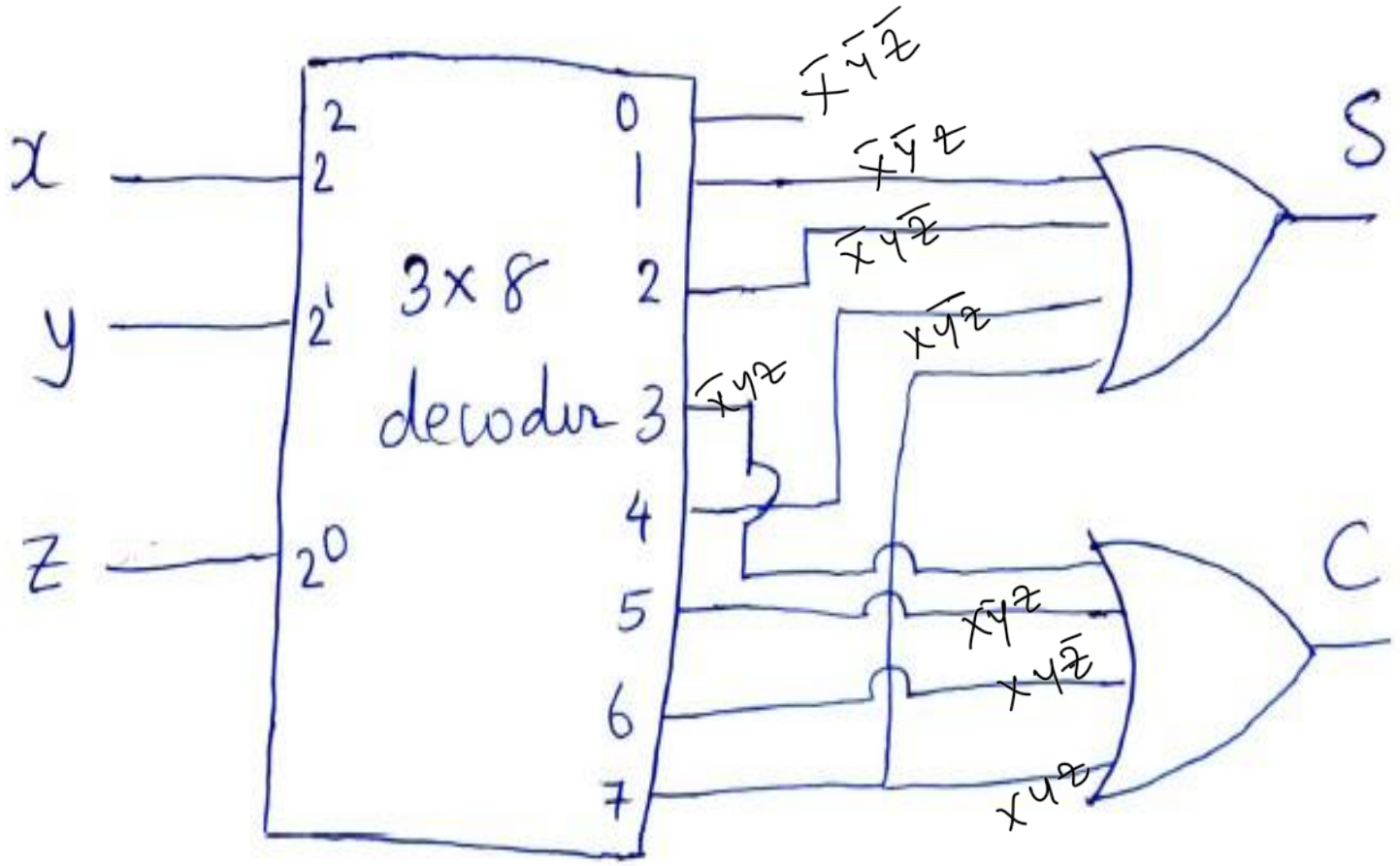
Explanation

- When w = 0, the top decoder is enabled.
- The bottom decoder outputs are all 0's.
- The top eight outputs generate minterms 0000 to 0111.
- When w = 1, the bottom decoder is enabled and it generates minterms 1000 to 1111. The top decoder outputs are all 0's now.

What else can we do with a decoder?

- A decoder can help realize some other functions.
- We show next how the sum and carry outputs of a full adder can be realized via a 3-to-8 decoder.

Full adder using a decoder



Explanation

- Recall the truth table of a full adder.
- From the truth table, we can observe that the sum and carry functions are given by:

$$S(x,y,z) = \sum(1,2,4,7)$$

$$C(x,y,z) = \sum(3,5,6,7)$$

Since there are three inputs (to a full adder) and a total of 8 minterms, we need a 3-to-8 decoder.

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- C. Roth and L.L. Kinney, Fundamentals of Logic Design, Gamma Publications, 2021